P1144R10
std::is_trivially_relocatable

Published Proposal, 2024-02-14

Author:
Arthur O'Dwyer

Audience:
LEWG, EWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

We define a new verb, "relocate," equivalent to a move and a destroy. For many C++ types, this is implementable "trivially," as a single `memcpy`. We propose a standard trait so library writers can detect such types. We propose a compiler rule that propagates trivial relocatability among Rule-of-Zero types. Finally, we propose a standard syntax for a user-defined type (e.g. `boost::container::static_vector`) to warrant to the implementation that it is trivially relocatable.

Table of Contents

§ 1. Changelog

- R10 (pre-Tokyo 2024):
  - Added implementation experience with [Abseil], [Amadeus], [HPX], [ParlayLib], [StdxError], and [Thrust]. Added §5.2 "Relevance of `operator=`".
  - Wording for `ExecutionPolicy` overloads of the specialized algorithms. Explain why there are no `ranges` overloads.
  - Moved "trivially relocatable class" wording into [class.prop], and increased the similarity between `[[trivially_relocatable]]` and `[[no_unique_address]]`.
- R9 (pre-Kona 2023):
  - Added feature-test macros `__cpp_impl_trivially_relocatable` and `__cpp_lib_trivially_relocatable`.
  - Added §3.1 "Design non-goals" and comparison to [P2785R3].
  - Replaced Appendix C "Examples of non-trivially relocatable class types" with a shorter summary.
  - Removed Appendix E "Open questions" as basically redundant with §5.3 "Confusing interactions with `std::pmr`". See "P1144 PMR koans" (June 2023).
- R8 (pre-Varna 2023):
  - dcl.attr.trivreloc: Removed "If a type `T` is declared `trivially_relocatable`, and `T` is either not move-constructible or not destructible, the program is ill-formed." This should have been removed in R7 along with "move-constructible, destructible."
  - meta.unary.prop: Adjusted the precondition of `is_trivially_relocatable_v` to match `is_trivially_copyable_v`. Removed `is_relocatable_v`, `is_nothrow_relocatable_v` for insufficient motivation (but kept `concept relocatable` because it expresses semantic requirements).
  - Marked `std::relocate` as `[[nodiscard]]`; changed its return type to `remove_cv_t<T>`.
  - Rewrote §3 "Design goals."
- R7 (post-Issaquah 2023):
  - Added `std::uninitialized_relocate_backward`, the building block of `vector::insert`.

- Removed "move-constructible, destructible" from the requirements in basic.types.general. Now trivially relocatable types, like trivially copyable types, can be non-relocatable.
- Added table §2.1 directly comparing this proposal to Folly, BSL, and Qt.
- Added straw poll results from Issaquah; removed much background and discussion (leaving only notes for the interested reader to consult [P1144R6]).

§ 2. Introduction and motivation

Given an object type `T` and memory addresses `src` and `dst`, the phrase "***relocate*** a `T` from `src` to `dst`" means "*move-construct* `dst` from `src`, and then immediately *destroy* the object at `src`."

Any type which is both move-constructible and destructible is ***relocatable***. A type is ***nothrow relocatable*** if its relocation operation is noexcept, and a type is ***trivially relocatable*** if its relocation operation is trivial (which, just like trivial move-construction and trivial copy-construction, means "tantamount to `memcpy`").

In practice, almost all relocatable types are trivially relocatable: `std::unique_ptr<int>`, `std::vector<int>`, `std::string` (on libc++ and MSVC), `std::list<int>` (on MSVC). Examples of non-trivially relocatable types include `std::list<int>` (on libstdc++ and libc++) and `std::string` (on libstdc++). See Appendix C: Examples of non-trivially relocatable class types and [InPractice].

P1144 allows the library programmer to ***warrant*** to the compiler that a resource-management type is trivially relocatable. Explicit warrants are rarely needed because the compiler can infer trivial relocatability for Rule-of-Zero types. See § 4.5 Trivially relocatable type [basic.types.general].

The most important thing about P1144 relocation is that it is backward compatible and does not break either API or ABI. P1144 intends simply to legalize the well-understood tricks that many industry codebases already do (see [BSL], [Folly], [Deque], [Abseil]), not to change the behavior of any existing source code (except to speed it up), nor to require major work from standard library vendors.

§ 2.1. Who optimizes on this?

The following optimizations are possible according to P1144's notion of trivial relocatability. Here's who does these optimizations in practice:

	vector realloc	type erasure	fixed-cap move	vector `erase`	vector `insert`	`rota`
Arthur's libc++ `std::is_trivially_relocatable_v<T>`	`vector`	no	N/A	`vector` (Godbolt)	`vector` (Godbolt)	yes, u (Godb
[BSL] `bslmf::IsBitwiseMoveable<T>`	`vector`	no	?	`vector`	`vector`	`Arra` unuse
[Folly] `folly::IsRelocatable<T>`	`fbvector`	no	`small_vector`	`fbvector`	`fbvector`	N/A
[Qt] `QTypeInfo<T>::isRelocatable`	`QList`	`QVariant`	`QVarLengthArray`	`QList`	`QList`	`q_ro`
[Abseil] `absl::is_trivially_relocatable<T>`	no	N/A	partial in `inlined_vector`	proposed for `inlined_vector`	no	N/A
[Amadeus] `amc::is_trivially_relocatable<T>`	`Vector`	N/A	`StaticVector`	`Vector`	`Vector`	N/A

§ 2.1.1. Contiguous reallocation

Trivial relocation helps anywhere we do the moral equivalent of `realloc`, such as in `vector<R>::reserve`.

[Bench] (presented at C++Now 2018) shows a 3x speedup on `vector<unique_ptr<int>>::resize`. This Reddit thread demonstrates a similar 3x speedup using the online tool Quick-Bench.

2.1.2. Moving in-place/SBO type-erased types like `any` and `function`

Trivial relocation can de-duplicate the code generated by type-erasing wrappers like `any`, `function`, and `move_only_function`. For these types, a *move* of the wrapper object is implemented in terms of a *relocation* of the contained object. (E.g., libc++'s `std::any`.) In general, the *relocate* operation must have a different instantiation for each different contained type `C`, leading to code bloat. But every trivially relocatable `C` of a given size can share the same instantiation.

2.1.3. Moving fixed-capacity containers like `static_vector` and `small_vector`

The move-constructor of `fixed_capacity_vector<R,N>` can be implemented naïvely as an element-by-element *move* (leaving the source vector's elements in their moved-from state), or efficiently as an element-by-element *relocate* (leaving the source vector empty). For detailed analysis, see [FixedCapacityVector].

`boost::container::static_vector<R,N>` currently implements the naïve element-by-element-move strategy, but after LEWG feedback, [P0843R5] `static_vector` does permit the faster relocation strategy.

2.1.4. Contiguous insert and erase

Trivial relocation can be used anywhere we shift a contiguous range left or right, such as in `vector::erase` (i.e., destroy the erased elements and "close the window" by memmoving left) and `vector::insert` (i.e., "make a window" by memmoving right and then construct the new elements in place). [Folly]'s `fbvector` does this optimization; see `fbvector::make_window`. Bloomberg's [BSL] also does this optimization.

2.1.5. Swap

Given a reliable way of detecting trivial relocatability, we can optimize any routine that uses the moral equivalent of `std::swap`, such as `std::rotate`, `std::partition`, or `std::sort`.

Optimizing `std::swap` produces massive code-size improvements for all swap-based algorithms, including `std::sort` and `std::rotate`. See [CppNow] @19:56–21:22, and see this Godbolt. This Quick-Bench shows a 25% speedup in `std::rotate` when it is allowed to use bitwise swap on a Rule-of-Zero type.

2.2. Assertions, not assumptions

Some data structures might reasonably assert the trivial relocatability of their elements, just as they sometimes assert the stronger property of trivial *copyability* today. This might help them guarantee reasonable performance, or guarantee exception-safety.

For an example of how `std::is_trivially_relocatable` is being used this way in practice, see [StdxError].

2.3. Eliminate manual warrants, improve safety

Many real-world codebases contain templates which require trivial relocatability of their template parameters, but cannot today *verify* trivial relocatability. For example, until 2012 [Folly] required the programmer to warrant the trivial relocatability of any type stored in a `folly::fbvector`:

```
class Widget {
    std::vector<int> lst_;
};

folly::fbvector<Widget> vec;  // FAIL AT COMPILE TIME for lack of warrant
```

This merely encouraged the programmer to add the warrant and continue. An incorrect warrant was discovered only at runtime, via undefined behavior. See [CppNow] @27:26–31:47; for real-world examples see Folly issues #889, #35, #316, #498.

```
class Gadget {
    std::list<int> lst_;
};
// sigh, add the warrant on autopilot
template<> struct folly::IsRelocatable<Gadget> : std::true_type {};

folly::fbvector<Gadget> vec;  // CRASH AT RUNTIME due to fraudulent warrant
```

> NOTE: Folly's `fbvector` was patched in December 2012 to accept both trivially relocatable and non-trivially relocatable types, in line with `std::vector`. Since then, the effect of an incorrect warrant remains the same — UB and crash — but a missing warrant simply disables the optimization.

If this proposal is adopted, then Folly can start using `static_assert(std::is_trivially_relocatable_v<T>)` resp. `if constexpr (std::is_trivially_relocatable_v<T>)` in the implementation of `fbvector`, and the programmer can stop writing explicit warrants. Finally, the programmer can start writing assertions of correctness, which aids maintainability and can even find real bugs. Example:

```
class Widget {
    std::vector<int> lst_;
};
static_assert(std::is_trivially_relocatable_v<Widget>);  // correctly SUCCEEDS

class Gadget {
    std::list<int> lst_;
};
static_assert(std::is_trivially_relocatable_v<Gadget>);  // correctly ERRORS
```

The improvement in user experience and safety in real-world codebases ([Folly], [BSL], [Qt], [HPX], [Amadeus], [Abseil], etc.) is the most important benefit to be gained by this proposal.

§ 3. Design goals

Briefly: We want to support all five of the following use-cases (Godbolt).

```
static_assert(std::is_trivially_relocatable_v<std::unique_ptr<int>>); // #1

struct RuleOfZero { std::unique_ptr<int> p_; };
static_assert(std::is_trivially_relocatable_v<RuleOfZero>); // #2

struct [[trivially_relocatable]] RuleOf3 {
    RuleOf3(RuleOf3&&);
    RuleOf3& operator=(RuleOf3&&);
    ~RuleOf3();
};
static_assert(std::is_trivially_relocatable_v<RuleOf3>); // #3

struct [[trivially_relocatable]] Wrap0 {
    boost::container::static_vector<int, 10> v_;
    static_assert(!std::is_trivially_relocatable_v<decltype(v_)>);
        // it's not annotated, but we know it's actually trivial
};
static_assert(std::is_trivially_relocatable_v<Wrap0>); // #4

struct [[trivially_relocatable]] Wrap3 {
    Wrap3(Wrap3&&);
```

```
    Wrap3& operator=(Wrap3&&);
    ~Wrap3();
    int i_;
    boost::interprocess::offset_ptr<int> p_ = &i_;
    static_assert(!std::is_trivially_relocatable_v<decltype(p_)>);
        // it's not trivial, but we preserve our invariant correctly
};
static_assert(std::is_trivially_relocatable_v<Wrap3>); // #5
```

§ 3.1. Non-goals

NOTE: This section was new in P1144R9.

We propose special support for trivially relocatable types, but no particular support for types that are relocatable in other ways. The two most-frequently-asked scenarios are:

- "Never-empty types," e.g. `gsl::not_null<unique_ptr<int>>`. Such types are not movable (because by design they never enter a moved-from state), but they are swappable and could in theory be relocatable (but not via move-and-destroy, since they cannot be moved). P1144 supports the physical possibility that `is_trivially_relocatable_v<T> && !is_relocatable_v<T>`, but keeps the status quo w.r.t. library support for such types; e.g. you can't `resize`, `insert`, or `erase` a `vector` of such types. P1144's `std::relocate_at` rejects such types.
- Types that are "efficiently but non-trivially relocatable," e.g. libc++'s `tuple<string, list<int>>`, where the ideal relocation operation would trivially relocate the string and move-and-destroy the list; this is not trivial, but still faster than move-and-destroy'ing the whole `tuple`. P1144 keeps the status quo w.r.t. such types, and doesn't provide any way to optimize their relocation. See "Cheaply relocatable, but not trivially relocatable" (October 2018).

The most promising active proposal for "non-trivial relocation" is [P2785R3]. It proposes a "relocation constructor" like this:

```
struct A {
    A(A) = default;
};
```

which the compiler deduces is trivial iff all of `A`'s members are trivially relocatable. This solves both of the above "non-goal" scenarios. However, [P2785R3] fails to support our positive goals `Wrap0` and `Wrap3`, which are trivially relocatable despite having some *non-trivial* members. In other words, P1144 is forward-compatible with (does not close the door to) [P2785R3]; and vice versa, adopting [P2785R3] would not solve two of P1144's five advertised use-cases. WG21 might well want to adopt both proposals. But P1144 solves "the 99% problem"; P1144 might not leave enough performance on the table to motivate the further adoption of [P2785R3].

Notably, P1144R10 is almost entirely a subset of [P2785R3]; the only significant difference is that [P2785R3] doesn't propose the `[[trivially_relocatable]]` warrant itself. (P2785 proposes that to make a type trivially relocatable, you `=default` its relocation constructor. P1144 can't do that, and anyway wants to support `Wrap0` and `Wrap3`.)

P1144	P2785R3
`[[trivially_relocatable]]`	
	`T(T) = default;`
	`T& operator=(T) = default;`
QoI `[[trivially_relocatable]]` on STL containers	Mandate relocation ctors for all STL containers
	`T t = reloc u;` (ends `u`'s lifetime)
`std::is_trivially_relocatable_v<T>`	`std::is_trivially_relocatable_v<T>`
`concept std::relocatable<T>`	
`std::relocate_at(psrc, pdst);`	`std::construct_at(pdst, reloc *psrc);`
`T t = std::relocate(psrc);`	`T t = std::destroy_relocate(psrc);`
`FwdIt std::uninitialized_relocate(InIt, InIt, FwdIt)`	`FwdIt std::uninitialized_relocate(InIt, InIt, FwdIt)`
`pair<InIt, FwdIt> std::uninitialized_relocate_n(InIt, Size, FwdIt)`	`pair<InIt, FwdIt> std::uninitialized_relocate_n(InIt, Size, FwdIt)`

`Bidi2 std::uninitialized_relocate_backward(` `  Bidi1, Bidi1, Bidi2)`	
	`std::relocate_t`, `std::relocate` tag type
	`vector<T>::relocate_out(` `  const_iterator, const_iterator, OutIt)` etc.
	`T queue<T>::pop(relocate_t)` etc.

§ 4. Proposed wording for C++26

The wording in this section is relative to the current working draft.

> NOTE: There is no difficulty in changing the attribute syntax to a contextual-keyword syntax; the only difference is aesthetic. We can defer that decision to the last minute. This wording is patterned on the precedent set by `[[no_unique_address]]`.

> NOTE: Our feature-test macros follow the pattern set by `__cpp_impl_coroutine`+`__cpp_lib_coroutine` and `__cpp_impl_three_way_comparison`+`__cpp_lib_three_way_comparison`.

§ 4.1. Predefined macro names [cpp.predefined]

Add a feature-test macro to the table in [cpp.predefined]:

```
__cpp_impl_three_way_comparison   201907L
__cpp_impl_trivially_relocatable  YYYYMML
__cpp_implicit_move               202207L
```

§ 4.2. Header `<version>` synopsis [version.syn]

Add a feature-test macro to [version.syn]/2:

```
#define __cpp_lib_transparent_operators   201510L // freestanding, also in <memory>, <functional>
#define __cpp_lib_trivially_relocatable   YYYYMML // freestanding, also in <memory>, <type_traits>
#define __cpp_lib_tuple_element_t         201402L // freestanding, also in <tuple>
```

§ 4.3. Relocation operation [defns.relocation]

Add a new section in [intro.defs]:

> **relocation operation [defns.relocation]**
>
> the homogeneous binary operation performed by `std::relocate_at`, consisting of a move construction immediately followed by a destruction of the source object

§ 4.4. Trivially relocatable class [class.prop]

> NOTE: For the "if supported" wording, compare [dcl.attr.nouniqueaddr]/2 and [cpp.cond]/5.

Modify [class.prop] as follows:

1. A *trivially copyable class* is a class:

- that has at least one eligible copy constructor, move constructor, copy assignment operator, or move assignment operator ([special], [class.copy.ctor], [class.copy.assign]),
- where each eligible copy constructor, move constructor, copy assignment operator, and move assignment operator is trivial, and
- that has a trivial, non-deleted destructor ([class.dtor]).

2. A *trivial class* is a class that is trivially copyable and has one or more eligible default constructors ([class.default.ctor]), all of which are trivial. [*Note:* In particular, a trivially copyable or trivial class does not have virtual functions or virtual base classes. — *end note*]

x. A *trivially relocatable class* is a class:

- where no eligible copy constructor, move constructor, copy assignment operator, move assignment operator, or destructor is user-provided,
- which has no virtual member functions or virtual base classes,
- all of whose members are either of reference type or of trivially relocatable type ([basic.types.general]), and
- all of whose base classes are of trivially relocatable type;

or a class that is declared with a `trivially_relocatable` attribute with value `true` ([dcl.attr.trivreloc]) if that attribute is supported by the implementation ([cpp.cond]).

3. A class `S` is a *standard-layout class* if it: [...]

§ 4.5. Trivially relocatable type [basic.types.general]

Add a new section in [basic.types.general]:

9. Arithmetic types ([basic.fundamental]), enumeration types, pointer types, pointer-to-member types ([basic.compound]), `std::nullptr_t`, and *cv*-qualified versions of these types are collectively called *scalar types*. Scalar types, trivially copyable class types ([class.prop]), arrays of such types, and *cv*-qualified versions of these types are collectively called *trivially copyable types*. Scalar types, trivial class types ([class.prop]), arrays of such types, and *cv*-qualified versions of these types are collectively called *trivial types*. Scalar types, standard-layout class types ([class.prop]), arrays of such types, and *cv*-qualified versions of these types are collectively called *standard-layout types*. Scalar types, implicit-lifetime class types ([class.prop]), array types, and *cv*-qualified versions of these types are collectively called *implicit-lifetime types*.

x. Trivially copyable types, trivially relocatable class types ([class.prop]), arrays of such types, and *cv*-qualified versions of these types are collectively called *trivially relocatable types*.

[*Note:* For a trivially relocatable type, the relocation operation ([defns.relocation]) as performed by, for example, `std::swap_ranges` or `std::vector::reserve`, is tantamount to a simple copy of the underlying bytes. —*end note*]

[*Note:* It is intended that most standard library types be trivially relocatable types. —*end note*]

10. A type is a *literal type* if it is: [...]

§ 4.6. Trivially relocatable attribute [dcl.attr.trivreloc]

NOTE: For the "Recommended practice" wording, compare [dcl.attr.nouniqueaddr]/2.

Add a new section after [dcl.attr.nouniqueaddr]:

1. The *attribute-token* `trivially_relocatable` specifies that a class type's relocation operation has no visible side-effects other than a copy of the underlying bytes, as if by the library function `std::memcpy`. It may be applied to the definition of a class. It shall appear at most once in each *attribute-list*. An *attribute-argument-clause* may be present and, if present, shall have the form

`( constant-expression )`

The *constant-expression* shall be an integral constant expression of type `bool`. If no *attribute-argument-clause* is present, it has the same effect as an *attribute-argument-clause* of `(true)`.

2. If any definition of a class type has a `trivially_relocatable` attribute with value *V*, then each definition of the same class type shall have a `trivially_relocatable` attribute with value *V*. No diagnostic is required if definitions in different translation units have mismatched `trivially_relocatable` attributes.

3. If a class type is declared with the `trivially_relocatable` attribute, and the program relies on observable side-effects of its relocation other than a copy of the underlying bytes, the behavior is undefined.

4. *Recommended practice:* The value of a *has-attribute-expression* for the `trivially_relocatable` attribute should be `0` for a given implementation unless this attribute can cause a class type to be trivially relocatable ([class.prop]).

4.6.1. `__has_cpp_attribute` entry [cpp.cond]

Add a new entry to the table of supported attributes in [cpp.cond]:

```
noreturn              200809L
trivially_relocatable YYYYMML
unlikely              201803L
```

4.7. `relocatable` concept [concept.relocatable]

Add a new section after [concept.copyconstructible]:

`relocatable` concept [concept.relocatable]

```
template<class T>
  concept relocatable = move_constructible<T>;
```

1. If `T` is an object type, then let `rv` be an rvalue of type `T`, `lv` an lvalue of type `T` equal to `rv`, and `u2` a distinct object of type `T` equal to `rv`. `T` models `relocatable` only if

- After the definition `T u = rv;`, `u` is equal to `u2`.
- `T(rv)` is equal to `u2`.
- If the expression `u2 = rv` is well-formed, then the expression has the same semantics as `u2.~T(); ::new ((void*)std::addressof(u2)) T(rv);`
- If the definition `T u = lv;` is well-formed, then after the definition `u` is equal to `u2`.
- If the expression `T(lv)` is well-formed, then the expression's result is equal to `u2`.
- If the expression `u2 = lv` is well-formed, then the expression has the same semantics as `u2.~T(); ::new ((void*)std::addressof(u2)) T(lv);`

NOTE: We intend that a type may be relocatable regardless of whether it is copy-constructible; but, if it is copy-constructible then copy-and-destroy must have the same semantics as move-and-destroy. We intend that a type may be relocatable regardless of whether it is assignable; but, if it is assignable then assignment must have the same semantics as destroy-and-copy or destroy-and-move. The semantic requirements on assignment help us optimize `vector::insert` and `vector::erase`. `pmr::forward_list<int>` satisfies `relocatable`, but it models `relocatable` only when all relevant objects have equal allocators.

4.8. Type trait `is_trivially_relocatable` [meta.unary.prop]

Add a new entry to Table 47 in [meta.unary.prop]:

Template	Condition	Preconditions
`template<class T> struct is_trivially_copyable;`	T is a trivially copyable type ([basic.types.general])	`remove_all_extents_t<T>` shall be a complete type or *cv* `void`.
`template<class T> struct is_trivially_relocatable;`	T is a trivially relocatable type ([basic.types.general])	`remove_all_extents_t<T>` shall be a complete type or *cv* `void`.
`template<class T> struct is_standard_layout;`	T is a standard-layout type ([basic.types.general])	`remove_all_extents_t<T>` shall be a complete type or *cv* `void`.

§ 4.9. `relocate_at` and `relocate`

NOTE: These functions have both been implemented in my libc++ fork; for `relocate`, see this Godbolt and [StdRelocateIsCute]. My implementation of their "as-if-by-memcpy" codepaths relies on Clang's `__builtin_memmove`; vendors can use any vendor-specific means to implement them. The wording also deliberately permits a low-quality implementation with no such codepath at all. See [CppNow] @45:23–48:39.

Add a new section after [specialized.destroy]:

`relocate` [specialized.relocate]

```
template<class T>
T *relocate_at(T* source, T* dest);
```

1. *Mandates:* T is a complete non-array object type.

2. *Effects:* Equivalent to:

```
struct guard { T *t; ~guard() { destroy_at(t); } } g(source);
return ::new (voidify(*dest)) T(std::move(*source));
```

except that if T is trivially relocatable ([basic.types.general]), side effects associated with the relocation of the value of `*source` might not happen.

```
template<class T>
[[nodiscard]] remove_cv_t<T> relocate(T* source);
```

3. *Mandates:* T is a complete non-array object type.

4. *Effects:* Equivalent to:

```
remove_cv_t<T> t = std::move(source);
destroy_at(source);
return t;
```

except that if T is trivially relocatable ([basic.types]), side effects associated with the relocation of the object's value might not happen.

§ 4.10. Nothrow bidirectional iterator [algorithms.requirements]

Modify [algorithms.requirements] as follows:

- If an algorithm's template parameter is named `ForwardIterator`, `ForwardIterator1`, `ForwardIterator2`, or `NoThrowForwardIterator`, the template argument shall meet the *Cpp17ForwardIterator* requirements ([forward.iterators]) if it is required to be a mutable iterator, or model `forward_iterator` ([iterator.concept.forward]) otherwise.
- If an algorithm's template parameter is named `NoThrowForwardIterator`, the template argument is also required to have the property that no exceptions are thrown from increment, assignment, or comparison of, or indirection through, valid iterators.
- If an algorithm's template parameter is named `BidirectionalIterator`, `BidirectionalIterator1`, ~~or~~ `BidirectionalIterator2`, or `NoThrowBidirectionalIterator`, the template argument shall meet the *Cpp17BidirectionalIterator* requirements ([bidirectional.iterators]) if it is required to be a mutable iterator, or model `bidirectional_iterator` ([iterator.concept.bidir]) otherwise.
- If an algorithm's template parameter is named `NoThrowBidirectionalIterator`, the template argument is also required to have the property that no exceptions are thrown from increment, decrement, assignment, or comparison of, or indirection through, valid iterators.

§ 4.11. `uninitialized_relocate`, `uninitialized_relocate_n`, `uninitialized_relocate_backward` [uninitialized.relocate]

> NOTE: Compare to [uninitialized.move] and [alg.copy]. The *Remarks* allude to blanket wording in [specialized.algorithms.general]/2.

> NOTE: I don't propose `ranges::uninitialized_relocate`, because if it worked like `ranges::uninitialized_move` then it would take two ranges (of possibly different lengths). On success, it would relocate-out-of only `distance(OR)` elements of `IR`; but if an exception were thrown, it would destroy all the elements of `IR`. That's a bad contract. Therefore, we omit the `ranges` overloads until someone presents a suitable design.

Modify [memory.syn] as follows:

```
template<class InputIterator, class NoThrowForwardIterator>
  NoThrowForwardIterator uninitialized_move(InputIterator first,                  // freestanding
                                            InputIterator last,
                                            NoThrowForwardIterator result);
template<class ExecutionPolicy, class ForwardIterator, class NoThrowForwardIterator>
  NoThrowForwardIterator uninitialized_move(ExecutionPolicy&& exec,               // see [algorithms.parall
                                            ForwardIterator first, ForwardIterator last,
                                            NoThrowForwardIterator result);
template<class InputIterator, class Size, class NoThrowForwardIterator>
  pair<InputIterator, NoThrowForwardIterator>
    uninitialized_move_n(InputIterator first, Size n,                             // freestanding
                         NoThrowForwardIterator result);
template<class ExecutionPolicy, class ForwardIterator, class Size,
         class NoThrowForwardIterator>
  pair<ForwardIterator, NoThrowForwardIterator>
    uninitialized_move_n(ExecutionPolicy&& exec,                                  // see [algorithms.parall
                         ForwardIterator first, Size n, NoThrowForwardIterator result);

namespace ranges {
  template<class I, class O>
    using uninitialized_move_result = in_out_result<I, O>;                        // freestanding
  template<input_iterator I, sentinel_for<I> S1,
           nothrow-forward-iterator O, nothrow-sentinel-for<O> S2>
    requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>>
      uninitialized_move_result<I, O>
        uninitialized_move(I ifirst, S1 ilast, O ofirst, S2 olast);               // freestanding
  template<input_range IR, nothrow-forward-range OR>
    requires constructible_from<range_value_t<OR>, range_rvalue_reference_t<IR>>
      uninitialized_move_result<borrowed_iterator_t<IR>, borrowed_iterator_t<OR>>
        uninitialized_move(IR&& in_range, OR&& out_range);                        // freestanding

  template<class I, class O>
    using uninitialized_move_n_result = in_out_result<I, O>;                      // freestanding
  template<input_iterator I,
           nothrow-forward-iterator O, nothrow-sentinel-for<O> S>
    requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>>
      uninitialized_move_n_result<I, O>
        uninitialized_move_n(I ifirst, iter_difference_t<I> n,                    // freestanding
                             O ofirst, S olast);
}

template<class InputIterator, class NoThrowForwardIterator>
  NoThrowForwardIterator uninitialized_relocate(InputIterator first,
                                                InputIterator last,
                                                NoThrowForwardIterator result);  // freestanding
template<class ExecutionPolicy, class ForwardIterator, class NoThrowForwardIterator>
  NoThrowForwardIterator uninitialized_relocate(ExecutionPolicy&& exec,           // see [algorithms.parall
                                                ForwardIterator first, ForwardIterator last,
                                                NoThrowForwardIterator result);

template<class InputIterator, class Size, class NoThrowForwardIterator>
  pair<InputIterator, NoThrowForwardIterator>
    uninitialized_relocate_n(InputIterator first, Size n,
                             NoThrowForwardIterator result);                      // freestanding
template<class ExecutionPolicy, class ForwardIterator, class Size,
         class NoThrowForwardIterator>
  pair<ForwardIterator, NoThrowForwardIterator>
    uninitialized_relocate_n(ExecutionPolicy&& exec,                              // see [algorithms.parall
                             ForwardIterator first, Size n, NoThrowForwardIterator result);

template<class BidirectionalIterator, class NoThrowBidirectionalIterator>
  NoThrowBidirectionalIterator
    uninitialized_relocate_backward(BidirectionalIterator first, BidirectionalIterator last,
                                    NoThrowBidirectionalIterator result);         // freestanding
template<class ExecutionPolicy, class BidirectionalIterator, class NoThrowBidirectionalIterator>
  NoThrowBidirectionalIterator
    uninitialized_relocate_backward(ExecutionPolicy&& exec,
                                    BidirectionalIterator first, BidirectionalIterator last,
                                    NoThrowBidirectionalIterator result);         // freestanding

template<class NoThrowForwardIterator, class T>
  void uninitialized_fill(NoThrowForwardIterator first,                           // freestanding
                          NoThrowForwardIterator last, const T& x);
```

Add a new section after [uninitialized.move]:

> **`uninitialized_relocate` [uninitialized.relocate]**
>
> ```
> template<class InputIterator, class NoThrowForwardIterator>
> NoThrowForwardIterator uninitialized_relocate(InputIterator first, InputIterator last,
>                                               NoThrowForwardIterator result);
> ```
>
> 1. *Effects:* Equivalent to:
>
> ```
> try {
>   for (; first != last; ++result, (void)++first) {
>     ::new (voidify(*result))
>       typename iterator_traits<NoThrowForwardIterator>::value_type(std::move(*first));
>     destroy_at(addressof(*first));
>   }
>   return result;
> } catch (...) {
>   destroy(++first, last);
>   throw;
> }
> ```
>
> except that if the iterators' common value type is trivially relocatable, side effects associated with the relocation of the object's value might not happen.
>
> 2. *Remarks:* If an exception is thrown, all objects in both the source and destination ranges are destroyed.
>
> ```
> template<class InputIterator, class Size, class NoThrowForwardIterator>
>   pair<InputIterator, NoThrowForwardIterator>
>     uninitialized_relocate_n(InputIterator first, Size n, NoThrowForwardIterator result);
> ```
>
> 3. *Effects:* Equivalent to:
>
> ```
> try {
>   for (; n > 0; ++result, (void)++first, --n) {
>     ::new (voidify(*result))
>       typename iterator_traits<NoThrowForwardIterator>::value_type(std::move(*first));
>     destroy_at(addressof(*first));
>   }
>   return {first, result};
> } catch (...) {
>   destroy_n(++first, --n);
>   throw;
> }
> ```
>
> except that if the iterators' common value type is trivially relocatable, side effects associated with the relocation of the object's value might not happen.
>
> 4. *Remarks:* If an exception is thrown, all objects in both the source and destination ranges are destroyed.
>
> ```
> template<class BidirectionalIterator, class NoThrowBidirectionalIterator>
>   NoThrowBidirectionalIterator
>     uninitialized_relocate_backward(BidirectionalIterator first, BidirectionalIterator last,
>                                     NoThrowBidirectionalIterator result);
> ```
>
> 5. *Effects:* Equivalent to:
>
> ```
> try {
>   for (; last != first; ) {
>     --last;
>     --result;
>     ::new (voidify(*result))
>       typename iterator_traits<NoThrowBidirectionalIterator>::value_type(std::move(*last));
>     destroy_at(addressof(*last));
>   }
>   return result;
> } catch (...) {
>   destroy(first, ++last);
>   throw;
> }
> ```
>
> except that if the iterators' common value type is trivially relocatable, side effects associated with the relocation of the object's value might not happen.
>
> 6. *Remarks:* If an exception is thrown, all objects in both the source and destination ranges are destroyed.

§ 5. Rationale and alternatives

§ 5.1. Attribute `[[maybe_trivially_relocatable]]`

My Clang implementation, currently available on Godbolt, supports both `[[trivially_relocatable]]` and another attribute called `[[clang::maybe_trivially_relocatable]]`, as suggested by John McCall in 2018 and also explored by P2786R0 in 2023. Where `[[trivially_relocatable]]` is a "sharp knife" that always cuts what you aim it at, `[[maybe_trivially_relocatable]]` is a "dull knife" that refuses to cut certain materials.

- `[[maybe_trivially_relocatable]]` fails to support § 3 Design goals examples #4 and #5.
- See "Should the compiler sometimes reject a `[[trivially_relocatable]]` warrant?" (2023-03-10).
- See P1144R4 §6.2.

In Issaquah (February 2023), P2786R0 suggested a "dull knife" design similar to `[[clang::maybe_trivially_relocatable]]`. EWGI took a three-way straw poll on `[[trivially_relocatable]]` versus `[[maybe_trivially_relocatable]]`, with an inconclusive 7–5–6 vote (the author of P1144 voting "For" and the three representatives of P2786 voting "Against"; i.e. the vote sans authors was 6–5–3).

§ 5.2. Relevance of `operator=`

Here's how current implementations define "is trivially relocatable":

- Mainline Clang `__is_trivially_relocatable(T)`: Trivially move-constructible and trivially destructible, or has `[[clang::trivial_abi]]`
- Arthur's Clang (P1144) `__is_trivially_relocatable(T)`: Trivially copyable, or has `[[clang::trivial_abi]]`, or has `[[trivially_relocatable]]`
- Facebook [Folly]'s `folly::IsRelocatable<T>`: `is_trivially_copyable<T>` or has member typedef `T::IsRelocatable`
- Bloomberg [BSL]'s `bslmf::IsBitwiseMoveable<T>`: Has a conversion to `bslmf::NestedTraitDeclaration<T, bslmf::IsBitwiseMoveable, true>`, or has a conversion to `bslmf::NestedTraitDeclaration<T, bslmf::IsBitwiseCopyable, true>`, or `is_trivially_copyable<T>`, or `sizeof(T) == 1`
- Google [Abseil]'s `absl::is_trivially_relocatable<T>`: `is_trivially_copyable<T>`, or `__is_trivially_relocatable(T)` on Linux Clang (not Windows or Apple); after #1625 it'll be "`is_trivially_copyable<T>`"
- [Amadeus] AMC's `amc::is_trivially_relocatable<T>`: `is_trivially_copyable<T>` or has member typedef `T::trivially_relocatable`
- Nvidia [Thrust]'s `thrust::is_trivially_relocatable<T>`: `is_trivially_copyable<T>`, or `proclaim_trivially_relocatable` is specialized for `T`
- [HPX]'s `hpx::experimental::is_trivially_relocatable<T>`: `is_trivially_copyable<T>`, or the trait is specialized for `T`
- [ParlayLib]'s `parlay::is_trivially_relocatable<T>`: currently "`is_trivially_move_constructible_v<T> && is_trivially_destructible_v<T>`," but after #67 it'll be "`is_trivially_copyable<T>`, or the trait is specialized for `T`"

Notice that all library implementors (except ParlayLib trunk and Abseil on Linux Clang due to the Clang builtin's current behavior) agree that a type with trivial move-constructor, trivial destructor, *and non-trivial assignment operator* is considered non-trivially relocatable. That is, the assignment operator is relevant!

P1144 preserves that behavior, and justifies it by pointing out that relocation can replace assignment in `vector::erase` and `std::swap`, among other algorithms. Existing codebases (most notably [Folly], [Amadeus], and [BSL]) have written a lot of code that branches on `is_trivially_relocatable`, taking an optimized codepath when `is_trivially_relocatable` and an unoptimized codepath otherwise. P1144 carefully proposes a definition for `std::is_trivially_relocatable` that keeps those codebases safe, and that never reports a type `T` as `is_trivially_relocatable` when it's not physically safe to use `T` with such an optimized codepath.

Here's a worked example showing why [BSL] pays attention to the assignment operator:

```
#include <bsl_vector.h>
using namespace BloombergLP::bslmf;

struct T : NestedTraitDeclaration<T, IsBitwiseMoveable> {
    int i_;
    T(int i) : i_(i) {}
    T(const T&) = default;
    void operator=(const T&) noexcept { puts("Called operator="); }
    ~T() = default;
```

```
};
int main() {
    bsl::vector<T> v = {1,2};
    v.erase(v.begin());
      // prints nothing; bsl::vector::erase is optimized
}

struct U {
    int i_;
    U(int i) : i_(i) {}
    U(const U&) = default;
    void operator=(const U&) noexcept { puts("Called operator="); }
    ~U() = default;
};
int main() {
    bsl::vector<U> v = {1,2};
    v.erase(v.begin());
      // prints "Called operator="; BSL believes that a non-defaulted
      // assignment operator makes U non-trivially relocatable.
      // P1144 agrees. std::is_trivially_relocatable_v<U>
      // must remain false, so as not to change the
      // behavior of bsl::vector<U>.
}
```

§ 5.3. Confusing interactions with `std::pmr`

> NOTE: See "P1144 PMR koans" (June 2023) for additional analysis of the problem cases.

> NOTE: This section was added in P1144R5, revised in R7, R8, and R10. Here I assume libc++, where `std::string` is trivially relocatable.

P1144 treats move as an optimization of copy; thus we assume that it always ought to be safe to replace "copy and destroy" with "move and destroy" (and thus with "relocate"). This is informed by Arthur's experience standardizing "implicit move" in P1155 (C++20) and P2266 (C++23). "Implicit move" affects `pmr` types because their copy differs from their move. Example:

```
struct ByValueSink {
    ByValueSink(std::pmr::vector<int> v) : v_(std::move(v)) {}
    std::pmr::vector<int> v_;
};

ByValueSink f(std::pmr::memory_resource *mr) {
    auto v = std::pmr::vector<int>(mr);
    return v;
      // C++17, Clang 12-: Copies (changes allocator)
      // C++20, GCC, Clang 13+: Moves (preserves allocator)
}
```

We didn't care about this when fiddling with implicit move; P1144 implicitly believes we shouldn't care about it now. (This position might be debatable, but it's definitely part of the worldview that led to P1144's specific proposed semantics, so it's good to have this background in mind.)

Some types invariably model `relocatable`; that is, their assignment is naturally tantamount to construct-and-destroy. But for `pmr` types and polymorphic types, assignment is *not* tantamount to construct-and-destroy, *except* under certain conditions. For `pmr` types, the condition is "both objects have the same allocator." For polymorphic types, the condition is "both objects have the same dynamic type."

```
std::vector<Derived> dv = { ~~~ };
dv.erase(dv.begin());
  // Every object in the vector certainly has the same dynamic type.

std::pmr::vector<std::pmr::string> pv = { ~~~ };
pv.erase(pv.begin());
  // Every string in the vector certainly has the same allocator.

Derived da[5] = { ~~~ };
```

```
std::rotate(da, da+2, da+5);
  // Every object in the contiguous range certainly has the same dynamic type.

std::pmr::string pa[5] = { ~~~ };
std::rotate(pa, pa+2, pa+5);
  // Objects in the contiguous range might have different allocators?
```

P1144 aims to permit efficient insertion and erasure in `std::vector`. Example:

```
std::vector<std::string> vs(501);
vs.erase(vs.begin());
```

This `erase` requires us to shift down 500 `std::string` objects. We can do this by 500 calls to `string::operator=` followed by one call to `~string`, or by one call to `~string` followed by a `memmove`, as in [BSL] (also [Folly], [Amadeus]; see table §2.1). We want to permit BSL's implementation. That's why the definition of `concept relocatable` in §4.8 places semantic requirements on `T`'s assignment operators (if they exist) as well as on `T`'s constructors and destructors.

If `pmr::string` is considered trivially relocatable, this will trickle down into all Rule-of-Zero types with a `pmr::string` member. (Godbolt.)

```
static_assert(std::is_trivially_relocatable_v<std::pmr::string>);
  // Before: not true, of course
  // After: suppose this is true
struct A {
  std::pmr::string ps;
};
static_assert(std::is_trivially_relocatable_v<A>);
  // After: then this will also be true, by the Rule of Zero
std::vector<A> v;
v.push_back({std::pmr::string("one", &mr1)});
v.push_back({std::pmr::string("two", &mr2)});
v.erase(v.begin());
  // Before: Well-defined behavior, acts as-if by assignment
  // After: Do we somehow make this a precondition violation?
assert(v[0].ps.get_allocator().resource() == &mr1);
  // Before: Succeeds
  // After: Might fail (and on a quality implementation, *will* fail)
```

If we (1) advertise `pmr::string` as `is_trivially_relocatable`; (2) propagate trivial relocatability in the core language as both P1144 and P2786 propose to do; and (3) optimize vector insert and erase for trivially relocatable types; then we inevitably arrive here. Arthur's solution is to impose preconditions on user-defined types used within certain parts of the STL: Just as their destructors are forbidden to (dynamically) throw exceptions, and their copy constructors are forbidden to (dynamically) make things that aren't copies, their assignment operators ought to be forbidden to (dynamically) violate `concept relocatable`'s semantic requirements.

The proximate conclusion, as far as P1144 is concerned, is that `pmr::string` should not be warranted trivially relocatable by any library vendor *unless* they're willing to place preconditions on how the user can use it in STL containers and algorithms (which probably wouldn't be conforming, before further evolution in this space).

The further evolution is out of scope for P1144, but see [P2959R0] for further elaboration of the problem, and [P3055R1] for a proposed solution.

§ 6. Acknowledgements

Thanks to Pablo Halpern for [N4158], to which this paper bears a striking resemblance —including the meaning assigned to the word "trivial," and the library-algorithm approach to avoiding the problems with "lame duck objects" discussed in the final section of [N1377]. See discussion of N4034 at Rapperswil (June 2014) and discussion of N4158 at Urbana (November 2014).

Significantly different approaches to this problem have previously appeared in Rodrigo Castro Campos's [N2754], Denis Bider's [P0023R0] (introducing a core-language "relocation" operator), and Niall Douglas's [P1029R3] (treating trivial relocatability as an aspect of move-construction in isolation, rather than an aspect of the class type as a whole).

A less different approach is taken by Mungo Gill & Alisdair Meredith's [P2786R2]. [P2814R1] compares P2786R0 against P1144R8.

Thanks to Elias Kosunen, Niall Douglas, John Bandela, and Nicolas Lesser for their feedback on early drafts of P1144R0. Thanks to Jens Maurer for his feedback on P1144R3 at Kona 2019, and to Corentin Jabot for championing P1144R4 at Prague 2020.

Thanks to Nicolas Lesser and John McCall for their review comments on the Clang implementation [D50119].

Many thanks to Matt Godbolt for allowing me to install my Clang fork on Compiler Explorer (godbolt.org). See also [Announcing].

Thanks to Howard Hinnant for appearing with me on [CppChat], and to Jon Kalb and Phil Nash for hosting us.

Thanks to Marc Glisse for his work integrating a "trivially relocatable" trait into GNU libstdc++ (see [Deque]) and for answering my questions on GCC bug 87106.

Thanks to Dana Jansens for her contributions re overlapping and non-standard-layout types (see [Subspace]), to Alisdair Meredith for our extensive discussions during the February 2023 drafting of P2786R0, to Giuseppe D'Angelo and Thiago Maceira for contributing the Qt entries in table §2.1, and to Giuseppe D'Angelo for extensive review comments and discussion.

Thanks to Charles Salvia (`stdx::error`), Isidoros Tsaousis-Seiras (HPX), Orvid King (Folly), Daniel Anderson (Parlay), and Derek Mauro (Abseil) for their work integrating P1144 into their respective libraries.

§ Appendix A: Straw polls

§ Polls taken at EWGI at Issaquah on 2023-02-10

Arthur O'Dwyer presented [P1144R6]. Alisdair Meredith presented P2786R0 (which proposed a `[[maybe_trivially_relocatable]]`-style facility, and expressed it as a contextual keyword instead of an attribute). EWGI took the following straw polls (as well as polls on attribute syntax and on both papers' readiness for EWG).

	SF	F	N	A	SA
The problem presented in P1144/P2786 is worth solving.	10	8	0	0	0
The problem being introduced in P1144/P2786 should be solved in a more general way instead of as proposed.	3	0	5	6	4
The annotation should "trust the user" as in P1144R6's `[[trivially_relocatable]]` ("sharp knife"), instead of diagnosing as in P1144R6's `[[clang::maybe_trivially_relocatable]]` and P2786R0's `trivially_relocatable` ("dull knife"). Three-way poll.	—	7	5	6	—

§ Polls taken at EWGI at Prague on 2020-02-13

Corentin Jabot championed P1144R4. EWGI discussed P1144R4 and Niall Douglas's [P1029R3] consecutively, then took the following straw polls (as well as a poll on the attribute syntax).

	SF	F	N	A	SA
We believe that P1029 and P1144 are sufficiently different that they should be advanced separately.	7	3	2	0	0
EWGI is ok to have the spelling as an attribute with an expression argument.	3	5	1	1	0

EWGI thinks the author should explore P1144 as a customizable type trait.	*0*	*0*	*0*	*9*	*2*
Forward P1144 to EWG.	*1*	*3*	*4*	*1*	*0*

For polls taken September–November 2018, see [P1144R6].

Appendix B: Sample code

See [P1144R6]'s Appendix B for reference implementations of `relocate`, `relocate_at`, and P1144R6's version of the `uninitialized_relocate` library algorithm, plus a conditionally trivially relocatable `std::optional<T>`.

Appendix C: Examples of non-trivially relocatable class types

See [P1144R6]'s Appendix C for compilable examples of types that are *not* trivially relocatable, for each of the following reasons:

- Class contains pointer to self (e.g. libstdc++'s `std::string`)
- Allocated memory contains pointer to self (e.g. libstdc++'s `std::list`)
- Class invariant depends on `this` (e.g. `boost::interprocess::offset_ptr`)
- Program invariant depends on `this` (e.g. a global registry of `Widget` addresses)
- Polymorphic object effectively relies on offset-into-self (see also [Polymorphic])

Appendix D: Implementation experience

Core-language implementations

Clang trunk provides a compiler builtin `__is_trivially_relocatable(T)` (see [D114732]), which is largely the same as the trait proposed in this paper. There remain slight differences; the only major one is that Clang still incorrectly reports types with user-provided assignment operators (and, on Windows, destructors) as "trivially relocatable." See the recently fixed Clang #67498 and Clang #77091, as well as Abseil 6b4af24.

As of February 2024, Clang trunk has no equivalent of the `[[trivially_relocatable]]` attribute, so `__is_trivially_relocatable(T)` is true only when `T` is trivially copyable and/or trivial for purposes of calls. But Clang's current status is compatible with P1144, modulo the few unintentional differences listed in the previous paragraph.

Arthur's fork of Clang implements all of P1144 (since 2018), and has been kept up-to-date with the latest P1144. See it in action on godbolt.org, under the name "x86-64 clang (experimental P1144)."

Library implementations

Since 2018, Arthur maintains a fork of libc++ with a fully optimizing implementation of P1144. See it in action on godbolt.org, under the name "x86-64 clang (experimental P1144)." (Here is another example showing P1144's interaction with fancy allocators.)

Since 2023, Arthur maintains a fork of libstdc++ with a conforming implementation of P1144, although it omits some optimizations. For example, `std::relocate` never lowers to `memcpy`, and (as of February 2024) `vector` reallocation never relocates. See it in action on godbolt.org, under the name "x86-64 clang (experimental P1144)," by passing `-stdlib=libstdc++` on the command line.

Since at least December 2018, Nvidia's [Thrust] implements `is_trivially_relocatable`, but uses it only as a detail of device-to-device copy. No relevant algorithms or container optimizations are provided.

Since October 2020, Carnegie Mellon's [ParlayLib] implements `uninitialized_relocate{,_n}` in the `parlay` namespace.

Since 2021, [Amadeus] AMC provides perhaps the most comprehensively optimizing `Vector`, `SmallVector`, and `StaticVector` implementations, as well as P1144-compatible `uninitialized_relocate{,_n}` and `relocate_at` in the `amc` namespace.

Since September 2023, Stellar [HPX] implements `relocate_at` and `uninitialized_relocate{,_n}` in the `hpx::experimental` namespace. This was a GSoC project.

[Qt] implements `q_uninitialized_relocate_n` in the `QtPrivate` namespace, but (unlike P1144's `std::uninitialized_relocate_n`) does not support overlap; it optimizes to `memcpy`. Qt also provides `q_relocate_overlap_n`, which optimizes to `memmove`.

Since November 2018, libstdc++ optimizes `vector::reserve` for types that manually specialize `std::__is_bitwise_relocatable`. (Godbolt.) As of February 2024, the only libstdc++ library type for which `__is_bitwise_relocatable` has been specialized is `deque`. See [Deque].

[ParlayLib], [HPX], [Thrust], [Qt], [Folly], and [BSL] all provide "type traits" that are intended to be manually specialized by the client programmer. When compiled with a P1144-compliant compiler, [HPX] and [ParlayLib] do not require (in fact, they forbid) the client programmer to specialize these type traits.

See table §2.1 for more details on most of these library implementations.

§ References

§ Informative References

[Abseil]
Aaron Jacobs; et al.. *Abseil C++ Common Libraries*. March 2023. URL: https://github.com/abseil/abseil-cpp

[Amadeus]
Stephane Janel. *AMadeus (C++) Containers*. April 2021. URL: https://github.com/AmadeusITGroup/amc

[Announcing]
Arthur O'Dwyer. *Announcing "trivially relocatable"*. July 2018. URL: https://quuxplusone.github.io/blog/2018/07/18/announcing-trivially-relocatable/

[Bench]
Arthur O'Dwyer. *Benchmark code from "The Best Type Traits C++ Doesn't Have"*. April 2018. URL: https://github.com/Quuxplusone/from-scratch/blob/095b246d/cppnow2018/benchmark-relocatable.cc

[BSL]
Bloomberg. *bslmf::IsBitwiseMoveable: bitwise moveable trait metafunction*. 2013–2022. URL: https://github.com/bloomberg/bde/blob/962f7aa/groups/bsl/bslmf/bslmf_isbitwisemoveable.h#L8-L48

[CppChat]
Howard Hinnant; Arthur O'Dwyer. *cpp.chat episode 40: It works but it's undefined behavior*. August 2018. URL: https://www.youtube.com/watch?v=8u5Qi4FgTP8

[CppNow]
Arthur O'Dwyer. *Trivially Relocatable (C++Now 2019)*. May 2019. URL: https://www.youtube.com/watch?v=SGdfPextuAU

[D114732]
Devin Jeanpierre. *[clang] Mark trivial_abi types as trivially relocatable*. November 2021. URL: https://reviews.llvm.org/D114732

[D50119]
Arthur O'Dwyer; Nicolas Lesser; John McCall. *Compiler support for P1144R0 __is_trivially_relocatable(T)*. July 2018. URL: https://reviews.llvm.org/D50119

[Deque]
Marc Glisse. *Improve relocation ... (__is_trivially_relocatable): Specialize for deque*. November 2018. URL: https://github.com/gcc-mirror/gcc/commit/a9b9381580de611126c9888c1a6c12a77d9b682e

[FixedCapacityVector]
Arthur O'Dwyer. *P1144 case study: Moving a `fixed_capacity_vector`*. URL: https://quuxplusone.github.io/blog/2019/02/22/p1144-fixed-capacity-vector/

[Folly]
Facebook. *Folly documentation on "Object Relocation"*. URL: https://github.com/facebook/folly/blob/master/folly/docs/FBVector.md#object-relocation

[HPX]
Isidoros Tsaousis-Seiras. *Relocation Semantics in the HPX Library*. August 2023. URL: https://isidorostsa.github.io/gsoc2023/

[InPractice]
Arthur O'Dwyer. *What library types are trivially relocatable in practice?*. February 2019. URL: https://quuxplusone.github.io/blog/2019/02/20/p1144-what-types-are-relocatable/

[N1377]
Howard Hinnant; Peter Dimov; Dave Abrahams. *N1377: A Proposal to Add Move Semantics Support to the C++ Language*. September 2002. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2002/n1377.htm

[N2754]
Rodrigo Castro Campos. *N2754: TriviallyDestructibleAfterMove and TriviallyReallocatable (rev 3)*. September 2008. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2008/n2754.html

[N4158]
Pablo Halpern. *N4158: Destructive Move (rev 1)*. October 2014. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4158.pdf

[P0023R0]
Denis Bider. *P0023R0: Relocator: Efficiently Moving Objects*. April 2016. URL: http://open-std.org/JTC1/SC22/WG21/docs/papers/2016/p0023r0.pdf

[P0843R5]
Gonzalo Brito Gadeschi. *static_vector*. July 2022. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p0843r5.html

[P1029R3]
Niall Douglas. *P1029R3: move = bitcopies*. January 2020. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1029r3.pdf

[P1144R6]
Arthur O'Dwyer. *Object relocation in terms of move plus destroy*. June 2022. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p1144r6.html

[P2785R3]
Sébastien Bini; Ed Catmur. *Relocating prvalues*. June 2023. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2785r3.html

[P2786R2]
Mungo Gill; Alisdair Meredith. *Trivial relocatability options*. June 2023. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2786r2.pdf

[P2814R1]
Mungo Gill; Alisdair Meredith. *Trivial relocatability — comparing P2786 with P1144*. May 2023. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2814r1.pdf

[P2959R0]
Relocation within containers. October 2023. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2959r0.html

[P3055R1]
Arthur O'Dwyer. *Relax wording to permit relocation optimizations in the STL*. February 2024. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3055r1.html

[ParlayLib]
Daniel Anderson; Guy Blelloch. *ParlayLib, a toolkit for programming parallel algorithms on shared-memory multicore machines*. October 2020. URL: https://github.com/cmuparlay/parlaylib/blob/4579f39/include/parlay/relocation.h

[Polymorphic]
Arthur O'Dwyer. *Polymorphic types aren't trivially relocatable*. June 2023. URL: https://quuxplusone.github.io/blog/2023/06/24/polymorphic-types-arent-trivially-relocatable/

[Qt]
Qt Base. February 2023. URL: https://github.com/qt/qtbase/

[StdRelocateIsCute]
Arthur O'Dwyer. *std::relocate's implementation is cute*. May 2022. URL: https://quuxplusone.github.io/blog/2022/05/18/std-relocate/

[StdxError]
Charles Salvia. *Implementation of std::error as proposed by Herb Sutter in P0709R0*. October 2023. URL: https://github.com/charles-salvia/std_error/

[Subspace]
Dana Jansens. *Trivially Relocatable Types in C++/Subspace*. January 2023. URL: https://danakj.github.io/2023/01/15/trivially-relocatable.html

[Thrust]
Bryce Adelstein Lelbach; Michał Dominiak. *Nvidia Thrust*. December 2018. URL: https://github.com/NVIDIA/cccl/blob/main/thrust/thrust/type_traits/is_trivially_relocatable.h